RabbitMQ: from sockets to a broker you can trust

Delivery, failure paths, clustering and diagnosis with RabbitMQ 4.4

21 October 2026

Contents

Scope and prerequisites **5**

1 From sockets to a broker **6**
1.1 What moves out of your process 6
1.2 The vocabulary, from the wire upwards 7
1.3 Routing: predict where a message goes 7
1.4 Where the ZeroMQ intuition stops working 8
1.5 Exercise 1: predict the route 9

2 Hand-offs you can trust **9**
2.1 Two hand-offs, two acknowledgements 9
2.2 Publisher confirms 10
2.3 Consumer acknowledgements 11
2.4 Prefetch: the unacknowledged window 12
2.5 At least once means idempotent consumers 12
2.6 Long work, heartbeats and the consumer timeout 13
2.7 Exercise 2: trace responsibility 14
2.8 Exercise 3: review a publisher 14

3 The job pipeline **15**
3.1 Shape and ownership 15
3.2 Topology as data 15
3.3 The producer 17
3.4 The worker 18
3.5 Running it and reading the result 20
3.6 What the run proves, and what it does not 21
3.7 Exercise 4: predict a crash 21

4 Choosing queues and failure paths **22**
4.1 Three queue types 22
4.2 Poison messages and the delivery limit 23
4.3 Retry needs a delay, which RabbitMQ does not add . . . 24
4.4 Dead-lettering and what lands in `jobs.dead` 25
4.5 Who owns topology 26
4.6 Growing the example 27
4.7 Exercise 5: review a generated consumer 27

5 Running a cluster **28**

5.1 What is replicated, and what needs a majority 28
5.2 The lab cluster, and the mistake it exposes 29
5.3 Losing nodes 30
5.4 Partitions 32
5.5 Resource alarms and flow control 32
5.6 Upgrades 33
5.7 Observability 34
5.8 Exercise 6: plan a node restart 34

6 Evidence before edits **35**
6.1 Directing an investigation 35
6.2 Symptoms, causes and distinguishing evidence 35
6.3 Probes that mislead 37
6.4 A worked investigation 37
6.5 Recovery after partial failure 38
6.6 Exercise 7: diagnose from output 39

7 Keeping it safe and maintainable **39**
7.1 Users, vhosts and permissions 39
7.2 Secrets 40
7.3 TLS and exposure 40
7.4 Reading an inherited broker 41
7.5 Exercise 8: review a definitions change 42

8 Integrating C++ clients (optional) **42**
8.1 The message contract comes first 42
8.2 Library choices 43
8.3 The shape of a correct C++ consumer 43
8.4 Exercise 9: review a C++ producer 44

Worked answers **45**
Exercise 1 45
Exercise 2 45
Exercise 3 45
Exercise 4 46
Exercise 5 46
Exercise 6 47
Exercise 7 47
Exercise 8 47
Exercise 9 48

Field reference **48**

Example files, sources and verification **50**
Files 50
Versions and sources 50
What was and was not verified 50

Scope and prerequisites

This tutorial is about predicting what RabbitMQ will do with a message well enough to judge a design, an implementation and a diagnosis. Light use of a RabbitMQ client is assumed; brokerless messaging with ZeroMQ is the main point of comparison, and the places where that comparison misleads are stated. The material covers building new messaging, operating a small cluster, and diagnosing incidents. Python carries the running example; an optional chapter covers C++ clients.

Allow two to three hours. Read chapters 1–7 in order; chapter 8, on C++ clients, is an optional branch that needs chapters 2 and 4. Each chapter ends with an exercise; worked answers are collected near the end. Pause at each exercise before reading its answer. Commands make behaviour precise.

The target is **RabbitMQ 4.4.0** (official Docker image `rabbitmq:4.4.0-management`, Erlang 29.1.2 in that image), with Python 3.12 and **pika 1.4.4**. RabbitMQ 4.4.0 was released on 19–20 October 2026; the 4.3 series is the other community-supported series, until 31 January 2027. Several behaviours described here changed in 4.0 and 4.3, so much published material about RabbitMQ, and much of what a language model has learned from it, is stale. Release information, 4.4.0 notes, pika 1.4.4.

The protocol taught is AMQP 0-9-1, which pika and the common C and C++ clients speak. RabbitMQ 4.x also speaks AMQP 1.0 natively; that matters in a few places and is noted there. Statements marked **observed** come from running the examples on 21 October 2026 against 4.4.0; the final appendix and `verification.md` list exactly what ran. Documented guarantees are linked to their source. Recommendations are labelled as such.

The running example is a small job pipeline. The `examples/` directory contains the complete files; everything essential is reproduced here. A build guide is in `README.md`.

1 From sockets to a broker

1.1 What moves out of your process

In ZeroMQ, the interesting state lives in your processes. A `PUSH` socket holds outgoing messages in a per-peer pipe up to its high-water mark; a `PULL` peer that dies takes whatever it had buffered with it. Nobody acknowledges anything. If you need "this job ran", you build it: request/reply, sequence numbers, retries, a store.

RabbitMQ moves that state into a separate, stateful server. A **queue** is a buffer owned by the broker, not by any process. It survives the disconnection of every producer and consumer, and, if it is durable and replicated, the loss of a broker node. A message leaves the queue only when a consumer **acknowledges** it. Delivery and responsibility therefore become explicit protocol events, which is the main thing you gain, and the main thing you must now reason about.

Question	ZeroMQ	RabbitMQ
Where are undelivered messages?	In sender/receiver socket buffers	In a broker queue
What happens if the receiver dies?	Its buffered messages are lost	Unacknowledged messages are requeued
How does the sender know it worked?	It doesn't, unless you build replies	Publisher confirms
Who decides fan-out?	Socket pattern and topology in code	Exchanges and bindings on the broker
What applies back-pressure?	High-water mark per socket	Queue limits, alarms, flow control, prefetch

The cost is that the broker is now a stateful dependency with its own failure modes: it can block publishers, lose quorum, or hold a backlog that grows silently. Much of this tutorial is about those modes.

1.2 The vocabulary, from the wire upwards

- A **connection** is one TCP (or TLS) session to one broker node, with authentication, heartbeats and flow control. Connections are relatively expensive; a process normally keeps one or a few for its lifetime.
- A **channel** is a lightweight session multiplexed over a connection. Most protocol operations happen on a channel. Errors such as "queue arguments do not match" close the channel, not the connection; some errors close the whole connection. Delivery tags are numbered per channel.
- A **virtual host** (vhost) is a namespace for exchanges, queues, bindings, policies and permissions. It is the unit of isolation between applications sharing a broker.
- An **exchange** receives published messages and routes copies to zero or more queues. It stores nothing.
- A **binding** is a rule linking an exchange to a queue (or to another exchange), usually with a **binding key**.
- A **queue** stores messages and delivers them to consumers.
- A publisher supplies an exchange name, a **routing key** and properties such as `delivery_mode`, `message_id` and `content_type`.

The publisher never names a queue. That indirection is the key design tool: producers state *what* happened (`routing_key="work"`); the broker's topology decides *who* receives it.

1.3 Routing: predict where a message goes

The exchange type defines how routing keys match binding keys. Exchanges.

- **Direct**: copy to every queue whose binding key equals the routing key.
- **Fanout**: copy to every bound queue, ignoring the key. This is the closest analogue of `PUB/SUB`, except that each subscriber has its own queue, which buffers messages when the subscriber is away.
- **Topic**: keys are dot-separated words; in a binding key `*` matches exactly one word and `#` matches zero or more. `orders.*.created` matches `orders.eu.created` but not `orders`

`.created`. Since 4.4 a topic binding key may contain at most two `#` wildcards. 4.4.0 notes.

- **Headers**: routes by message header values rather than the key. Rarely needed; prefer topic unless the routing data genuinely is multi-dimensional.

Every vhost also has a **default exchange**, named by the empty string. Every queue is bound to it automatically, with its own name as binding key, so `basic_publish("", "jobs.work", ...)` delivers straight to the queue `jobs.work`. That convenience bypasses your topology, and with it any routing you later add.

Several queues bound to one exchange each receive a copy. Several consumers on one queue **compete**: each message goes to one of them. That is the RabbitMQ version of `PUSH`/`PULL` load distribution, with acknowledgements added.

Unroutable messages are dropped by default. If no binding matches, the exchange routes the message nowhere, and the publish still succeeds from the client's point of view; with publisher confirms enabled it is even *confirmed*. Two mechanisms change that. With the `mandatory` flag, the broker returns the message to the publisher (`basic.return`) before confirming it. An **alternate exchange**, configured by policy on the exchange, receives anything the exchange cannot route. Routing to an alternate exchange counts as routed for `mandatory`. Confirms, alternate exchanges.

Observed: publishing to the lab's `jobs` exchange with the mistyped key `wokr` and `mandatory=True` produced pika's `UnroutableError`; without `mandatory`, the same publish would have been confirmed and the message silently lost.

1.4 Where the ZeroMQ intuition stops working

- *"If the socket accepted it, it's on its way."* A RabbitMQ publish returning from the client library means only that bytes were written to a socket. Responsibility transfers at the confirm (chapter 2).
- *"Slow consumers make senders block."* RabbitMQ queues absorb backlog until a length limit, memory alarm or disk alarm intervenes. A backlog is normal operation until it is not, so you

must decide its bounds explicitly.
- *"Topology is in my code."* It can be, but the broker persists it. Two services declaring the same queue with different arguments now conflict at runtime (chapter 4).
- *"Restarting a process clears things."* Restarting a consumer redelivers its unacknowledged messages. Restarting the broker keeps durable queues and persistent messages.

1.5 Exercise 1: predict the route

A topic exchange `events` has three bindings: queue `audit` with `#`, queue `billing` with `invoice.*`, queue `eu` with `*.eu.#`. Predict which queues receive `invoice.eu.created`, `invoice.paid`, `refund.eu` and `invoice`. A producer then publishes `invoce.paid` (sic) to `events` with confirms but without `mandatory`. What does it observe, and where is the message?

Ask the agent: "For this producer, list every way a successfully returning publish can still result in no queue holding the message."

- *A good answer* names unroutable messages (missing or mistyped binding, wrong exchange, default-exchange publish to a misspelt queue), explains that confirms alone do not detect them, and proposes `mandatory` with a return handler or an alternate exchange plus monitoring of that exchange's queue. It also mentions length-limit overflow and TTL as later losses.
- *Warning signs*: "confirms guarantee the message reached a queue"; adding `mandatory` with no code that handles the return; or treating the default exchange as an ordinary routing choice.

2 Hand-offs you can trust

2.1 Two hand-offs, two acknowledgements

Follow responsibility for a single job:

1. The producer holds the job. If it crashes now, the job exists only in its own state.
2. The producer publishes. Until the broker **confirms**, the outcome is unknown: the message may or may not be enqueued.

3. The broker confirms. The broker is now responsible.
4. The broker delivers the message to a consumer. It is now **unacknowledged**: still owned by the queue, but invisible to other consumers.
5. The consumer acknowledges. The queue deletes the message.

Publisher confirms close the gap between 2 and 3; consumer acknowledgements close the gap between 4 and 5. Without both, "at least once" is wishful thinking. Confirms and acknowledgements.

2.2 Publisher confirms

A channel in confirm mode receives a `basic.ack` for every published message, identified by a sequence number. The documented meaning of that ack depends on the destination:

- For a **quorum queue**, the message is confirmed once a quorum of replicas has accepted it.
- For a **durable classic queue** and a persistent message, it is confirmed after it is written to disk.
- For an **unroutable** message, it is confirmed as soon as the exchange finds no route. With `mandatory`, the return arrives before the ack.
- A `basic.nack` means the broker did not accept the message: an internal error in the queue process, or a full queue whose overflow setting is `reject-publish` (chapter 4). Your code must treat it as "not accepted".

Confirms are asynchronous at the protocol level. pika's `BlockingConnection` hides that: after `channel.confirm_delivery()`, `basic_publish` waits for the ack, raising `NackError` or `UnroutableError` as appropriate. pika blocking adapter. Waiting per message is simple and slow; high-throughput publishers keep a window of outstanding confirms and track them by sequence number. That is a mechanical choice you can delegate, provided the code still knows which messages are unconfirmed when the connection drops.

An unconfirmed message is in an unknown state, not a lost one. Observed in the cluster lab: with two of three nodes stopped, a publish to the surviving node was accepted onto the socket but never confirmed; the producer was killed after 30 seconds. When the other

nodes returned, the "unconfirmed" message was in the queue. The same happened under a memory alarm on a single node. A producer that republishes every unconfirmed message after reconnecting is correct only if consumers tolerate duplicates.

A message is also only as durable as its properties and queue. Set `delivery_mode=2` (persistent) for messages that must survive a restart. Quorum queues and streams are always durable.

2.3 Consumer acknowledgements

A consumer chooses an acknowledgement mode when it subscribes:

- **Automatic** (`auto_ack=True` in pika, `no_ack` on the wire): the broker considers the message delivered as soon as it has written it to the socket. A consumer crash loses everything in flight. Use it only where loss is acceptable.
- **Manual**: the consumer calls one of
  - `basic_ack`: done; delete it.
  - `basic_reject` or `basic_nack` with `requeue=True`: put it back for another attempt. RabbitMQ returns it to its original position, or as near the head as possible.
  - `basic_reject` or `basic_nack` with `requeue=False`: dead-letter it if a dead-letter exchange is configured; otherwise discard it.

If a channel or connection closes with unacknowledged deliveries, the broker requeues them, with the `redelivered` flag set on the next delivery. An ack must be sent on the channel that received the delivery; acknowledging a tag on another channel is a protocol error that closes the channel. Confirms and acknowledgements.

`redelivered=True` means "this message may have been delivered before", not "this message was processed before". Observed: a worker with prefetch 10 crashed while handling job 3. Jobs 4 and 5 had already been sent to it, but not touched. All three came back with `redelivered=True`.

2.4 Prefetch: the unacknowledged window

`basic_qos(prefetch_count=N)` limits how many unacknowledged messages the broker will send to a consumer at once. In RabbitMQ, the classic `global=false` form applies the limit per consumer. Per-channel "global" QoS is a deprecated feature that 4.3 denies by default and quorum queues never supported. Consumer prefetch, 4.3.0 notes.

Prefetch is a tradeoff that needs evidence, not a convention:

- **Too low** (1) and each consumer waits a network round trip between messages; throughput suffers when jobs are short.
- **Too high** or unlimited (0) and one consumer hoards messages that idle consumers could process. Its crash returns the whole window at once, redelivered, and its memory holds them all.

A useful starting rule (a recommendation, not a documented default) is a window that covers the round-trip time at the consumer's processing rate, then measured. For slow, uneven jobs, small prefetch values distribute work more fairly.

2.5 At least once means idempotent consumers

Put the hand-offs together. The producer may publish twice (an unconfirmed message retried after reconnection). The broker may deliver twice (a consumer crashed after doing the work but before its ack reached the broker, or a consumer timeout, or a partition). RabbitMQ provides **at-least-once** delivery when both hand-offs are confirmed; it does not provide exactly-once processing. The consumer must make a repeated delivery harmless.

The usual technique is an **idempotency key**: the producer sets a stable `message_id` derived from the job, not from the attempt; the consumer records processed keys in the same transaction as the job's effect. Then, if the effect committed, the key is present and a redelivery is ignored; if it did not commit, neither did the key, and redelivery repeats the work. A key recorded in a separate transaction recreates the gap you were closing.

The lab's worker does this with SQLite: the "work" is inserting the key, so the effect and the record are trivially one transaction. In a real service the key belongs in the database the job modifies. Where the

effect is external, such as sending email or calling a payment API, you need that system's own idempotency mechanism, or you accept and bound duplicates.

2.6 Long work, heartbeats and the consumer timeout

Two timers watch slow consumers.

Heartbeats detect dead TCP peers; the broker's default proposed timeout is 60 seconds. In pika's `BlockingConnection`, your callback runs on the thread that services the connection. A job that blocks for longer than the heartbeat timeout stops heartbeats, and the broker closes the connection, which requeues the job that was nearly done. pika's documentation recommends doing long work on another thread and acknowledging through `connection.add_callback_threadsafe`, the only thread-safe method on a `BlockingConnection`. Heartbeats, pika blocking adapter. pika's development branch deprecates `BlockingConnection` in favour of a new thread-safe `Connection`; that change is not in 1.4.4, so check before upgrading pika.

The **consumer timeout** limits how long a delivery may stay unacknowledged. In 4.3 it moved into the queue implementations, and only quorum queues evaluate it; classic queues and streams never do. The broker's effective default in 4.4.0 is 1,800,000 ms (30 minutes), observed with `rabbitmqctl eval 'application:get_env(rabbit, consumer_timeout).'`. Observed on a quorum queue with a 5-second timeout: the broker cancelled the consumer after 5.0 s, the channel stayed open, and the message returned to the queue. Quorum queues. The quorum-queue page explains which: the broker cancels the consumer if the client advertises the `consumer_cancel_notify` capability, as pika does, and otherwise closes the channel with `PRECONDITION_FAILED`. A client library without that capability, such as some C or C++ clients, therefore sees a channel error instead of a cancel. For a worker this means a job slower than the timeout is redelivered to someone else, while the original worker silently stops receiving deliveries unless it handles the cancel.

2.7 Exercise 2: trace responsibility

A worker with manual acks and prefetch 5 receives jobs 1–5. It finishes job 1 and acks it, finishes job 2 and commits its database transaction, then the process is killed before acking job 2. Another worker is running. Which jobs does the second worker receive, with what `redelivered` flag, and what must its code do with job 2?

Ask the agent: "Walk me through each point where this worker can crash, and say for each whether the job is lost, duplicated, or neither."

- *A good answer* traces publish, confirm, delivery, effect commit and ack in order, places the idempotency key inside the effect's transaction, and identifies the crash between commit and ack as the duplicate case. It also covers consumer timeout and heartbeat loss for slow jobs.
- *Warning signs*: claims of exactly-once delivery; using `redelivered` to decide whether to skip work; acking before the effect commits "for throughput"; `auto_ack=True` in a job consumer.

2.8 Exercise 3: review a publisher

An agent proposes this to "make publishing reliable":

```
channel.confirm_delivery()
for job in jobs:
    while True:
        try:
            channel.basic_publish("jobs", "work", encode(job))
            break
        except pika.exceptions.AMQPError:
            channel = reconnect()   # also calls
 ↪  confirm_delivery()
```

List what it gets right and what it leaves unproven. What would you ask for before approving it?

3 The job pipeline

3.1 Shape and ownership

The example is the smallest system that exercises both hand-offs, retries and dead-lettering:

- A **producer** submits numbered jobs to the direct exchange `jobs` with routing key `work`, waiting for a confirm on each.
- **Workers** consume from the quorum queue `jobs.work` with manual acks and record each job in an idempotent ledger.
- Jobs that fail permanently, or that exceed the delivery limit, are dead-lettered through the fanout exchange `jobs.dlx` to the quorum queue `jobs.dead`, where a human or a separate tool decides what to do with them.

The broker's topology, users and permissions are not declared by the applications. They are loaded from a **definitions file** when the broker boots, and the application user has no permission to create or change them. That is a design decision, explained in chapter 4; the example makes it concrete.

```
producer
   |  publish to exchange "jobs", routing key "work"
   v
[jobs] --binding "work"--> (jobs.work) --> workers
                                  |
                                  |  rejected without requeue,
                                  |  or over the delivery limit
                                  v
                           [jobs.dlx] --> (jobs.dead)
```

3.2 Topology as data

The relevant part of `examples/broker/definitions.json` (users and permissions are shown in chapter 7):

```
"exchanges": [
  {"name": "jobs", "vhost": "jobs", "type": "direct", "durable":
   ↪  true,
   "auto_delete": false, "internal": false, "arguments": {}},
  {"name": "jobs.dlx", "vhost": "jobs", "type": "fanout",
   ↪  "durable": true,
```

```
    "auto_delete": false, "internal": false, "arguments": {}}
],
"queues": [
  {"name": "jobs.work", "vhost": "jobs", "durable": true,
   ↪  "auto_delete": false,
   "arguments": {"x-queue-type": "quorum"}},
  {"name": "jobs.dead", "vhost": "jobs", "durable": true,
   ↪  "auto_delete": false,
   "arguments": {"x-queue-type": "quorum"}}
],
"bindings": [
  {"source": "jobs", "vhost": "jobs", "destination":
   ↪  "jobs.work",
   "destination_type": "queue", "routing_key": "work",
    ↪  "arguments": {}},
  {"source": "jobs.dlx", "vhost": "jobs", "destination":
   ↪  "jobs.dead",
   "destination_type": "queue", "routing_key": "", "arguments":
    ↪  {}}
],
"policies": [
  {"name": "work-failures", "vhost": "jobs", "pattern":
   ↪  "^jobs\\.work$",
   "apply-to": "quorum_queues", "priority": 0,
   "definition": {"dead-letter-exchange": "jobs.dlx",
                  "dead-letter-strategy": "at-least-once",
                  "overflow": "reject-publish", "max-length":
                   ↪  10000,
                  "delivery-limit": 5}},
  {"name": "dead-letters", "vhost": "jobs", "pattern":
   ↪  "^jobs\\.dead$",
   "apply-to": "quorum_queues", "priority": 0,
   "definition": {"delivery-limit": -1}}
]
```

Notice what lives where. The queue's *type* is a declaration argument, fixed for the queue's lifetime. Everything that an operator might tune later (where failures go, how many attempts, the length bound) is in a **policy**, which can be changed on a running broker without redeploying anything. Chapter 4 explains each policy key.

`examples/broker/rabbitmq.conf` tells the node to import that file at boot:

```
definitions.import_backend = local_filesystem
definitions.local.path = /etc/rabbitmq/definitions.json
```

Definitions.

3.3 The producer

`examples/producer.py`, complete:

```
"""Submit jobs FIRST..FIRST+COUNT-1, waiting for a broker
 ↪  confirm for each.

Usage: python producer.py FIRST COUNT [ROUTING_KEY]
Exit status: 0 all confirmed, 2 unroutable, 3 nacked by the
 ↪  broker.
"""
import json
import sys

import pika
from pika.exceptions import NackError, UnroutableError

from common import parameters


def main():
    first, count = int(sys.argv[1]), int(sys.argv[2])
    routing_key = sys.argv[3] if len(sys.argv) > 3 else "work"
    with pika.BlockingConnection(parameters()) as connection:
        channel = connection.channel()
        channel.confirm_delivery()
        for n in range(first, first + count):
            message_id = f"job-{n}"
            properties = pika.BasicProperties(
                content_type="application/json",
                delivery_mode=pika.DeliveryMode.Persistent,
                message_id=message_id,
            )
            body = json.dumps({"job": n}).encode()
```

```
            try:
                # Blocks until the broker confirms or rejects
                ↪ this message.
                channel.basic_publish("jobs", routing_key, body,
↪ properties,
                                      mandatory=True)
            except UnroutableError:
                print(f"{message_id} unroutable: no queue bound
                ↪ for {routing_key!r}")
                return 2
            except NackError:
                print(f"{message_id} nacked: the broker did not
                ↪ take responsibility")
                return 3
            print(f"{message_id} confirmed")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Each property earns its place: `confirm_delivery` and `mandatory` close the publish hand-off and the routing hole; `Persistent` makes the message survive a broker restart; `message_id` is derived from the job, so a resubmission carries the same key. `common.py` builds the connection from `AMQP_URL`, defaulting to the lab's loopback broker and `app` user.

What the producer deliberately does not do: retry after a connection failure, or wait without a bound. Both appear in the exercises and in chapter 5, where a publish to a broker without quorum waits forever.

3.4 The worker

The core of `examples/worker.py`; argument parsing is omitted. Options such as `--poison`, `--flaky` and `--crash-before-ack` exist only to make delivery behaviour visible.

```
ledger = sqlite3.connect(args.ledger, isolation_level=None)
ledger.execute("CREATE TABLE IF NOT EXISTS done "
               "(message_id TEXT PRIMARY KEY, pid INTEGER, at
                ↪ REAL)")
```

```
with pika.BlockingConnection(parameters()) as connection:
    channel = connection.channel()
    channel.basic_qos(prefetch_count=args.prefetch)
    deliveries = channel.consume("jobs.work",
    ↪ inactivity_timeout=args.idle)
    for method, properties, body in deliveries:
        if method is None:  # idle for args.idle seconds
            break
        job = json.loads(body)["job"]
        count = (properties.headers or
    ↪ {}).get("x-delivery-count", 0)
        tag = f"{properties.message_id}
    ↪ redelivered={method.redelivered} count={count}"
        if job in args.poison:
            print(f"{tag} rejected (poison)")
            channel.basic_reject(method.delivery_tag,
    ↪ requeue=False)
            continue
        if job in args.flaky:
            print(f"{tag} requeued by {args.retry_verb}")
            if args.retry_verb == "reject":
                channel.basic_reject(method.delivery_tag,
    ↪ requeue=True)
            else:
                channel.basic_nack(method.delivery_tag,
    ↪ requeue=True)
            continue
        time.sleep(args.work_seconds)
        inserted = ledger.execute(
            "INSERT OR IGNORE INTO done VALUES (?, ?, ?)",
            (properties.message_id, os.getpid(),
    ↪ time.time())).rowcount
        print(f"{tag} {'recorded' if inserted else 'duplicate
         ↪ ignored'}")
        if job == args.crash_before_ack:
            print(f"{tag} crashing before ack")
            os._exit(1)
        channel.basic_ack(method.delivery_tag)
    channel.cancel()
```

The order inside the loop is the point: do the work, commit its record, then ack. A crash before the commit repeats the work; a crash between commit and ack produces a redelivery that the ledger recognises. The worker does not declare the queue: it assumes the topology exists, and fails loudly if it does not.

The jobs here take 0.1 s, so the single-threaded `BlockingConnection` loop is acceptable. A job that can approach the heartbeat timeout needs the worker-thread pattern from chapter 2.

3.5 Running it and reading the result

Runnable on a machine with Docker and Python 3.12, from `rabbitmq/examples/`:

```
python3 -m venv .venv && .venv/bin/pip install -r
 ↪  requirements.txt
./lab.sh     # destroys and recreates the lab broker, then runs
 ↪  every step
```

`lab.sh` starts the broker with `docker compose up --wait`, runs each experiment, and prints queue counters after each. Ports are published on `127.0.0.1` only. The observed results of steps 1–5, abridged and reformatted:

```
## 1. publish jobs 1-5 with confirms
job-1 confirmed  ...  job-5 confirmed
jobs.work  ready=5 unacked=0

## 2. publish with a mistyped routing key
job-6 unroutable: no queue bound for 'wokr'          exit=2

## 3. worker crashes after recording job 3, before acking it
job-1 redelivered=False count=0 recorded
job-2 redelivered=False count=0 recorded
job-3 redelivered=False count=0 recorded
job-3 redelivered=False count=0 crashing before ack
jobs.work  ready=3 unacked=0

## 4. a fresh worker drains the queue
job-3 redelivered=True count=1 duplicate ignored
job-4 redelivered=True count=1 recorded
```

```
job-5 redelivered=True count=1 recorded

## 5. the producer is rerun for jobs 1-5
job-1 redelivered=False count=0 duplicate ignored  ...  (all
 ↪  five ignored)
```

Read it against the model. Step 3 left three messages, not five: jobs 1 and 2 were acked. Step 4 shows the crash-before-ack duplicate being absorbed, and shows jobs 4 and 5 marked redelivered although no code ever looked at them; they were in the crashed worker's prefetch window. The quorum queue's `x-delivery-count` header went from 0 to 1 because the consumer's connection was lost (chapter 4 explains which events increment it). Step 5 shows that the broker does not deduplicate: five new messages with old `message_id`s were delivered and the ledger ignored them.

One operational detail matters when reading these results. Counters from `rabbitmqctl list_queues` for quorum queues are refreshed periodically, not per message. Immediately after five confirmed publishes they read zero; a few seconds later, five. A consumer count also lingered for a few seconds after a worker exited. `lab.sh` therefore sleeps before sampling. An agent diagnosing "messages vanished" from a single sample can be misled by this.

3.6 What the run proves, and what it does not

It shows the delivery semantics on one node with tiny jobs. It does not show throughput, the effect of large prefetch on memory, behaviour across node failure (chapter 5), or that a real service's effect and idempotency key share a transaction, which is the property that actually prevents double processing. A successful run with a ledger in a separate database from the effect would print exactly the same lines.

3.7 Exercise 4: predict a crash

Start with an empty queue and ledger. Publish jobs 1–10. Run `worker.py --prefetch 1 --crash-before-ack 4`, then a plain `worker.py`. Predict the second worker's output lines, including `redelivered` and `count`. How would the answer change with `--prefetch 10`?

4 Choosing queues and failure paths

4.1 Three queue types

RabbitMQ 4.x offers three kinds of queue, chosen per queue at declaration by `x-queue-type`, and immutable afterwards. Quorum, classic, streams.

	Classic	Quorum	Stream
Replication	None (mirroring removed in 4.0)	Raft, default 3 replicas	Replicated log; confirms once a majority of replicas has it
Consumption	Destructive	Destructive	Non-destructive; consumers choose an offset
Durability	Durable or exclusive	Always durable	Always durable
Poison handling	None	Delivery limit (default 20)	None
Dead-lettering	At most once	At most once or at least once	None
Good for	Exclusive, temporary, replies	Work queues and events that must survive node loss	Replay, many readers, large fan-out

Recommendations for this kind of work:

- Use **quorum queues** for anything whose loss you would investigate. They are the replicated, data-safe type, and the default you should expect an agent to choose.
- Use **classic queues** for exclusive or short-lived queues tied to one connection (per-client reply queues, temporary subscriptions), and for single-node brokers where replication is irrelevant.
- Use **streams** when consumers need to re-read history or start from an offset, which is closer to Kafka than to a work queue. Streams are an optional branch: they change the consumer

contract, so read the stream documentation before adopting them.

A plausible stale default: much example code declares queues with pika's defaults, `queue_declare("tasks")`, which is a transient, non-exclusive classic queue. That combination is a deprecated feature denied by default since 4.3. Observed on 4.4.0: the declaration closed the **whole connection** with `541 INTERNAL_ERROR - Feature transient_nonexcl_queues is deprecated`. A queue is either durable (survives restarts) or exclusive (lives with its connection); pick one deliberately.

Quorum queues also cannot be exclusive and do not support global QoS. If you need exactly one active consumer at a time, use the `x-single-active-consumer` argument: one consumer receives deliveries and another takes over if it disappears. Messages in a queue are FIFO per publishing channel, but requeues and priorities reorder them; if strict order matters, single active consumer (or a stream) is the tool, and it caps throughput at one consumer.

4.2 Poison messages and the delivery limit

A **poison message** is one that fails every time, for example because a malformed field makes the handler throw. With requeue-on-failure, it returns to the head of the queue and is retried forever, consuming the worker and delaying the jobs behind it.

Quorum queues count deliveries in the `x-delivery-count` header and dead-letter (or drop, if no dead-letter exchange is configured) a message redelivered more times than its `delivery-limit`. The default is 20 since 4.0; the lab sets 5. Since 4.4, changing the `delivery-limit` policy applies to existing quorum queues. Poison message handling, 4.4.0 notes.

The critical detail is **which client actions count**. The documentation's table for 4.3 and later is:

Increments `x-delivery-count`	Does not increment
AMQP 0-9-1 `basic.reject`	AMQP 0-9-1 `basic.nack`
Client crash or connection loss	Consumer timeout

Increments `x-delivery-count`	Does not increment
AMQP 1.0 `rejected`; `modified` with `delivery-failed=true`	Network partition; AMQP 1.0 `released`

When is delivery count incremented.

That asymmetry between `reject` and `nack` is easy to miss, and a model trained on older material will not know it. Observed in steps 6 and 8 of `lab.sh`:

```
## 6. poison job 21 and flaky job 22, retried with basic.reject
job-21 redelivered=False count=0 rejected (poison)
job-22 redelivered=False count=0 requeued by reject
job-22 redelivered=True count=1 requeued by reject
  ...
job-22 redelivered=True count=5 requeued by reject
jobs.work ready=0    jobs.dead ready=2

## 7. dead letters
job-21 rejected [('rejected', 1)]
job-22 delivery_limit [('delivery_limit', 1)]

## 8. flaky job 30 retried with basic.nack, stopped after 5 s
deliveries in 5 s: 842
job-30 redelivered=False count=0
job-30 redelivered=True count=0
```

With `reject`, job 22 was delivered six times (count 0 to 5) and then dead-lettered with reason `delivery_limit`. With `nack`, job 30 was delivered 842 times in five seconds (834 on a repeat run) with the count stuck at 0; in an unbounded run, one message was delivered more than 20,000 times in under two minutes. The delivery limit offers no protection against that loop.

4.3 Retry needs a delay, which RabbitMQ does not add

Even with `reject` and a delivery limit, immediate requeue retries at once. A transient failure such as a downstream service being briefly unavailable burns through five attempts in milliseconds, and the job

is dead-lettered for a failure that would have cleared in a minute. The options, in increasing complexity:

1. **Immediate requeue with a limit.** Right for failures that are really about this worker (an exception that another worker may not hit), and as a poison-message backstop. Cheap and built in.
2. **Delayed retry via a waiting queue.** The worker publishes the failed job, with confirms, to a retry queue whose messages expire after a delay and are dead-lettered back to the work exchange, then acks the original. This adds backoff, at the cost of another queue, a duplicate window between publish and ack, and attempt counting by the application (for example, via the `x-death` count). *Design sketch; not part of the tested lab.*
3. **Application-level scheduling.** Record the failure in the job's own store and resubmit later. Right when retries need business decisions.

This is a decision to retain. Ask which failures are transient, how long they last, and what happens to a job that exhausts its attempts.

4.4 Dead-lettering and what lands in `jobs.dead`

Messages are dead-lettered when they are rejected or nacked with `requeue=False`, when a per-message TTL expires, when a length limit drops them, or when a quorum queue's delivery limit is exceeded. Dead-lettering republishes the message to the configured exchange and adds an `x-death` header recording, per queue and reason, a count, time, exchange and routing keys, plus `x-first-death-reason` and related headers that are set once. Dead letter exchanges.

By default, dead-lettering is **at most once**: if the dead-letter target is unavailable, the message is lost. The lab's policy chooses **at least once** for quorum queues, which requires `dead-letter-strategy: at-least-once`, `overflow: reject-publish` (it does not work with the default `drop-head`), and a dead-letter exchange. The documentation also recommends a length limit. With `reject-publish`, a full queue refuses new publishes, which the publisher sees as a nack, instead of silently discarding its oldest messages. Observed with `max-length 1`: a classic queue nacked the second publish, while a quorum queue accepted two messages before nacking. Treat a quorum queue's length limit as approximate. At-least-once dead-lettering.

That last choice changes the producer's contract: it must now handle nacks. The lab producer exits with status 3. Whether a full queue should refuse new work or shed old work is a product decision, not a default to accept silently.

The dead-letter queue is not a solution; it is a place where problems wait. Somebody must monitor its depth, inspect messages, and decide whether to fix and replay, or discard. Replaying dead letters into the work queue without fixing the cause recreates the failure; replaying them without idempotent consumers can duplicate effects.

4.5 Who owns topology

There are three common ways to create exchanges, queues and bindings:

1. **Applications declare what they use** at startup. Easy to start; common in examples. Declarations are idempotent only if every declarer uses identical arguments. Observed: redeclaring `jobs.work` as a classic queue failed with `406 PRECONDITION _FAILED - inequivalent arg 'x-queue-type'`, which closes the channel. Two service versions with different arguments cannot run side by side.
2. **Definitions or infrastructure code own topology**; applications only publish and consume, with permissions that forbid declaring. The lab does this. Observed: the `app` user's attempt to declare `jobs.work` was refused with `403 ACCESS _REFUSED - configure access to queue 'jobs.work'`.
3. **A hybrid**: applications declare their own private or temporary queues, while shared topology is centrally owned.

Option 2 makes topology reviewable as data, prevents drift from ad hoc declarations, and supports least privilege, at the cost of a deployment step before a new consumer can run. Option 1 suits a single service owning a private broker. For services shared across teams, or with C++ and Python clients that must agree, the recommendation is option 2 or 3. This is a decision for you, not the agent.

Policies versus queue arguments. A policy matches queues (or exchanges) by name pattern and supplies optional settings: dead-lettering, limits, TTLs, delivery limit. At most one policy applies

to a queue: the highest-priority match, chosen nondeterministically between equal priorities. Queue arguments (`x-...`) are fixed at declaration. Where both set the same key, the client's arguments win, except for stream settings and a quorum queue's `overflow`, where the policy wins. **Operator policies**, set by administrators, are merged on top as guardrails; for most numeric limits the lower value applies (`target-group-size` is an exception: the greater wins). Policies can be changed at any time; arguments require recreating the queue. Recommendation: put only immutable choices in arguments (`x-queue-type`), and everything tunable in policies. An agent that hard-codes `x-dead-letter-exchange` in application code has made a later operational change into a migration. Policies, dead letter exchanges.

4.6 Growing the example

Pressures that justify more structure:

- **A second kind of consumer** (for example, an audit service that sees every job) means another queue bound to the exchange, not a second consumer on `jobs.work`. If the set of consumers will grow, a topic exchange with meaningful keys (`job.submitted.render`) costs nothing now.
- **Different services** belong in different vhosts if they must not see or interfere with each other's queues, with users per service.
- **Schema changes** need a version in the message (a header or field) and a period where consumers accept both versions. RabbitMQ does not inspect bodies; mismatches surface as consumer exceptions, which become poison messages.
- **Several environments** each get their own definitions; avoid a single broker hosting staging and production vhosts unless you accept that they share resource alarms (chapter 5).

4.7 Exercise 5: review a generated consumer

An agent writes this worker for a new service on the 4.4 cluster:

```
channel.queue_declare("thumbnails",
                      arguments={"x-dead-letter-exchange":
 ↪  "dead"})
channel.basic_qos(prefetch_count=0)
```

```
def handle(ch, method, properties, body):
    try:
        make_thumbnail(json.loads(body))
    except Exception:
        ch.basic_nack(method.delivery_tag, requeue=True)
        return
    ch.basic_ack(method.delivery_tag)

channel.basic_consume("thumbnails", handle)
channel.start_consuming()
```

The agent says "failures are retried, and the dead-letter exchange catches anything that keeps failing". Identify each problem, its observable consequence, and the change you would require.

Ask the agent: "What happens to a message that fails on every delivery, step by step, on RabbitMQ 4.4, and how would we see it in metrics?"

- *A good answer* distinguishes `reject` from `nack` for the delivery count, names the delivery limit and dead-letter target, explains that requeue is immediate, and points to redelivery rate and dead-letter queue depth as signals.
- *Warning signs*: relying on a delivery limit with `nack`; citing a classic queue `x-max-priority` or mirroring setting as current; requeue without any limit; a dead-letter queue nobody monitors.

5 Running a cluster

5.1 What is replicated, and what needs a majority

A RabbitMQ cluster is a set of Erlang nodes sharing one **metadata store** (users, vhosts, exchanges, bindings, queue definitions, policies) and hosting queues. Since 4.3, the metadata store is **Khepri**, a Raft-based store, and the older Mnesia store has been removed. Khepri needs a majority of nodes online to change metadata. Metadata store, 4.3.0 notes.

Queues are replicated independently:

- Each **quorum queue** is its own Raft group: a leader and followers on different nodes. A publish is confirmed when a majority of its members have written it. With three members it tolerates one failure; with two of three down it is unavailable.
- A **classic queue** lives on one node. If that node is down, the queue is unavailable; if its disk is lost, so are its messages.
- A **stream** is also replicated and, like a quorum queue, needs a majority of its replicas to confirm a publish.

Hence the documented sizing rules: use an odd number of nodes, three or five; two nodes are strongly discouraged, and four or six tolerate no more failures than three or five. Quorum queues default to three members, which the documentation calls the practical minimum; more than seven is not recommended. Clustering, quorum queue replication.

Nodes authenticate each other with a shared secret, the **Erlang cookie**. Any holder of the cookie who can reach the inter-node port can run arbitrary code on every node, so it is a root-equivalent credential (chapter 7).

5.2 The lab cluster, and the mistake it exposes

`examples/cluster/compose.yaml` runs three nodes, `rabbit1`–`rabbit3`, which discover each other from a fixed list in `examples/cluster/rabbitmq.conf`:

```
cluster_formation.peer_discovery_backend = classic_config
cluster_formation.classic_config.nodes.1 = rabbit@rabbit1
cluster_formation.classic_config.nodes.2 = rabbit@rabbit2
cluster_formation.classic_config.nodes.3 = rabbit@rabbit3

definitions.import_backend = local_filesystem
definitions.local.path = /etc/rabbitmq/definitions.json
```

Each node also imports the same definitions file at boot, which looks harmless. `examples/cluster/lab.sh` starts the cluster and immediately asks for the members of `jobs.work`. Observed, twice:

```
## 1. members of jobs.work after definitions were imported at
 ↪  boot
    rabbit@rabbit3 leader
```

The cluster had three running nodes, but the quorum queue had **one** member, on whichever node imported the definitions before the others had joined. A quorum queue's initial members are chosen at declaration from the nodes then in the cluster; members are not added later when nodes join, unless **continuous membership reconciliation** is enabled, and it is disabled by default. Membership reconciliation. Nothing in the logs or the client's behaviour revealed this. The queue worked perfectly, without any replication.

The lab repairs it explicitly, then checks:

```
rabbitmq-queues grow rabbit@rabbit1 all --vhost-pattern '^jobs$'
 ↪  --queue-pattern '.*'
# ...and likewise for rabbit2 and rabbit3
rabbitmq-queues quorum_status jobs.work --vhost jobs
```

```
## 2. grow every jobs queue onto every node
    rabbit@rabbit1 follower
    rabbit@rabbit2 follower
    rabbit@rabbit3 leader
```

The durable fixes are to import topology only once the cluster has formed (a deployment step that waits for all nodes, then imports), to enable membership reconciliation with a target group size, or both. The general lesson: check replication by asking the broker for membership, never by reading the declaration.

5.3 Losing nodes

The rest of `cluster/lab.sh` stops nodes and publishes through `rabbit1` (loopback port 5673). Observed:

```
## 3. stop rabbit3, publish jobs 100-102 through rabbit1
job-100 confirmed  job-101 confirmed  job-102 confirmed
 ↪  exit=0
    rabbit@rabbit1 leader
    rabbit@rabbit2 follower
    rabbit@rabbit3 noconnection

## 4. may rabbit2 be stopped now?
queue 'jobs.work' in vhost 'jobs' will become unavailable if
 ↪  node rabbit@rabbit2 stops
```

```
queue 'jobs.dead' in vhost 'jobs' will become unavailable if
 ↪  node rabbit@rabbit2 stops
rabbitmq_metadata will become unavailable if node rabbit@rabbit2
 ↪  stops
exit=69

## 5. stop rabbit2 as well, publish job 103 through rabbit1 (30
 ↪  s limit)
exit=124

## 6. restart rabbit2 and rabbit3, count messages
jobs.work   4
```

Step 3: with one node down, a new leader was elected and publishes were confirmed normally. Step 4: `check_if_node_is_quorum_critical` exited with status 69 and named what would become unavailable, including the metadata store. That is the check to run before stopping any node deliberately. rabbitmq-queues(8).

Step 5 is the instructive one. A separate instrumented run showed that the client **connected and opened a confirm-mode channel instantly** on the surviving node; it then waited for a confirm that never came, until `timeout` killed it. Nothing errored. Step 6: after quorum returned, the queue held four messages, so the unconfirmed job 103 was enqueued after all.

Consequences for design:

- A producer must bound its wait for confirms and treat a timeout as "unknown", which implies retry with the same `message_id`, which implies idempotent consumers. pika's blocking `basic_publish` has no timeout argument; a production publisher needs the asynchronous confirm pattern or a watchdog.
- A client connects to **one node**. When that node stops, its connections close and the client must reconnect elsewhere. pika's `BlockingConnection` accepts a list of connection parameters and tries them in order, but it does not reconnect for you after a connection is lost; reconnection, and re-establishing consumers, is application code. Alternatively, a TCP load balancer in front of the nodes gives one address.
- Health checks that only test "is the port open" will report the

minority node as healthy while it cannot accept a single message.

5.4 Partitions

A network partition splits the cluster into groups that cannot see each other. Before 4.3, operators chose a partition-handling strategy such as `pause_minority`. Since 4.3 there is no choice: the strategies were removed, and the `cluster_partition_handling` keys are accepted but ignored. Raft-based components decide availability: the majority side elects leaders and continues; minority-side members cannot commit. Messages may be delivered twice around a partition. Partitions, 4.3.0 notes. An agent recommending `cluster_partition_handling = pause_minority` for a 4.4 cluster is configuring nothing.

5.5 Resource alarms and flow control

RabbitMQ protects itself when memory or disk runs short. The defaults are a memory high watermark of 0.6 of available memory and a free disk limit of 50 MB (observed in 4.4.0's runtime configuration as `0.6` and `50000000`). When an alarm fires, the broker **blocks connections that publish** across the whole cluster; connections that only consume continue, so queues can drain. Clients that support it receive a `connection.blocked` notification. Alarms, memory, disk alarms.

Observed with the lab broker's watermark lowered to trigger the alarm: a publisher waited until killed after 25 seconds; `list_connections` showed its connection as `blocked`; a worker on another connection consumed normally; and after the alarm cleared, the message from an earlier killed publisher had been enqueued, the "unknown, not lost" pattern again.

Design rules that follow, documented as recommendations:

- Use **separate connections for publishing and consuming** in a process that does both. A shared connection that publishes is blocked, so its consumers stop acknowledging and the queue cannot drain.
- Give publishers a bound: pika's `blocked_connection_timeout` connection parameter raises `ConnectionBlockedTimeout` instead of waiting forever.

- Alert on alarms before they block production traffic. A 16 MiB default maximum message size is a separate limit that rejects oversized publishes outright.

Below the alarms, **credit flow** slows individual fast publishers by briefly blocking and unblocking their connections many times a second. A connection in state `flow` is being throttled; occasional flow is normal, persistent flow means the broker cannot keep up. Flow control.

5.6 Upgrades

An in-place upgrade must follow a path the upgrade guide documents. Some paths skip a series (4.0.x to 4.2.x, for example), so do not assume either "one series at a time" or "any jump works". 4.3.0 supports upgrades only from 4.2.x, and requires all stable feature flags, including `khepri_db`, to be enabled first. The 4.4.0 notes do not state its supported upgrade path explicitly; they strongly recommend moving to 4.3.7 or later first when also upgrading to Erlang 28 or later. Verify the path in the release notes of the exact target before planning. Upgrade, feature flags, 4.3.0 notes.

A rolling upgrade of a three-node cluster is, for each node in turn:

1. Check `rabbitmq-queues check_if_node_is_quorum_critical` on that node and proceed only on success. `rabbitmq-upgrade await_online_quorum_plus_one` is the waiting form of the same gate: it blocks until no quorum queue would lose its quorum if this node stopped.
2. `rabbitmq-upgrade drain` to move client connections and queue leadership away.
3. Stop, upgrade, start, and wait until the node has rejoined the cluster.
4. Only then move to the next node. Its step 1 check will not pass until the previous node's queue members are back online.

After the last node, `rabbitmq-queues rebalance all` spreads queue leaders across the cluster again. A mixed-version cluster is a transitional state during the upgrade, not a steady state. The alternative, a **blue-green** migration to a new cluster with consumers moved across and old queues drained, avoids in-place upgrade constraints at the cost of duplicate infrastructure and a migration plan.

It is the safer choice when jumping several series. For a .0 release such as 4.4.0, a cautious production plan waits for the first patch or two, or rehearses on a copy of production definitions.

5.7 Observability

The `rabbitmq_prometheus` plugin serves metrics on port 15692. By default `/metrics` returns aggregated metrics; per-queue metrics are available from `/metrics/per-object` or the filterable `/metrics /detailed`, and cost more at scale. Prometheus. The signals that map onto this tutorial's failure modes:

- Ready and unacknowledged message counts per important queue, and their trend, not their instantaneous value.
- Consumers per queue; zero consumers on a work queue is an incident.
- Redelivery rate; a high rate with flat throughput suggests a requeue loop.
- Dead-letter queue depth.
- Resource alarms, file descriptors, memory and disk headroom per node.
- Connections in `blocked` or `flow` state, and connection churn.
- Quorum queue members per queue, compared with the intended group size.

5.8 Exercise 6: plan a node restart

You inherit a three-node 4.4 cluster. `rabbitmq-queues quorum _status` shows that `orders` has members on `n1` and `n2` only; the other queues have three members. You must restart `n2` for a kernel update while `n1` is healthy and `n3` is healthy. Predict what `check_if_node_is _quorum_critical` on `n2` reports, what happens to `orders` producers during the restart if you proceed anyway, and what you would do first.

Ask the agent: "Before we restart this node, show me evidence that every queue and the metadata store keep a majority."

- *A good answer* runs `check_if_node_is_quorum_critical` on the node and reads its exit status, shows `quorum_status` for queues it is unsure of, explains the two-member case, and proposes

`rabbitmq-queues grow` (or membership reconciliation) before the restart.

- *Warning signs*: inferring safety from `cluster_status` alone (all nodes running says nothing about queue membership); using the declared group size as evidence; recommending partition-handling settings; restarting nodes in parallel "to save time".

6 Evidence before edits

6.1 Directing an investigation

When an agent is handed logs and access and asked to find the cause of an incident, it will produce a plausible story quickly. The reviewer's job is to check that the story is supported by evidence that would have looked different if the story were false. For a broker, the most useful evidence is usually **counters with time**, **connection and channel states**, **queue membership and leaders**, **alarm state**, and **the exact error code** a client received.

Before changing anything, decide which observation distinguishes the hypotheses. Before accepting a fix, decide what metric or output will prove it worked. Purging a queue, deleting a member, or restarting a node destroys evidence and sometimes data; such actions need an explicit human decision.

6.2 Symptoms, causes and distinguishing evidence

Symptom	Plausible causes	Evidence that distinguishes them
Queue backlog growing	No consumers; consumers too slow; consumers stuck holding prefetch; requeue loop	`consumers`, `messages_ready` versus `messages_unacknowledged`, ack rate, redelivery rate

Symptom	Plausible causes	Evidence that distinguishes them
Publishers hang	Resource alarm; quorum lost for the target queue; flow control; full queue with `reject-publish` and an unhandled nack	`rabbitmq-diagnostics alarms`, connection state `blocked/flow`, `quorum_status`, client exceptions
Messages "missing"	Unroutable (no `mandatory`); TTL expiry; `drop-head` overflow; `auto_ack` consumer crashed; dead-lettered	Unroutable or returned counts, dead-letter queue contents and `x-death` reasons, queue policies, consumer ack mode
Duplicate processing	Crash between effect and ack; consumer timeout; heartbeat loss; producer retry after an unknown confirm	`redelivered` and `x-delivery-count` on the duplicates, consumer cancel or connection close logs, producer retry logs
Worker idle while queue has messages	Consumer cancelled by consumer timeout; channel closed by an error; consumer on the wrong queue or vhost	Broker log for cancellations, `list_consumers`, client logs for channel close codes
High CPU, throughput flat	Hot requeue loop (`nack`), connection churn	Redelivery rate, `x-delivery-count` stuck at 0, connection open/close rate
Client error 406	Redeclare with different arguments; consumer timeout (`PRECONDITION_FAILED` on some paths); unknown delivery tag	The reply text, which names the cause

Symptom	Plausible causes	Evidence that distinguishes them
Client error 403	Permissions do not match the operation	The reply text names the operation and resource
Connection closed with 541	A denied deprecated feature, such as a transient non-exclusive queue	The reply text names the feature

6.3 Probes that mislead

Three observed traps in 4.4.0:

1. `check_local_alarms` **is a no-op.** Run with `-q`, as a health check would, it printed "reported no local alarms" and exited 0 while a memory alarm was blocking publishers on that very node. Without `-q` it prints "This command is DEPRECATED and is a no-op". `check_alarms` reported the alarm and exited 69. The behaviour is the same in 4.3.6; 4.2.9 still reports the alarm. The rabbitmq-diagnostics man page, checked the same day, still describes `check_local_alarms` as a working check: the documentation lags the binary, so test a health probe by inducing the failure it is meant to detect.
2. **Queue counters lag.** Quorum queue counters from `list _queues` were seconds behind confirmed publishes and consumer exits. Look at trends or repeat the sample.
3. `check_running` **and** `cluster_status` **say nothing about quorum.** In the cluster lab, all three were green while `jobs .work` had one member.

6.4 A worked investigation

Report: after a deploy of a new worker for the quorum queue `thumbnails`, which has a dead-letter exchange and the default delivery limit by policy, CPU on the broker and workers rose, `thumbnails` stopped draining, and no dead letters appeared.

A weak investigation looks at "queue not draining", concludes "workers

too slow", and scales workers. That would make it worse.

A sound one:

1. *Counters:* `messages_ready` is high and steady, `messages_unacknowledged` is small, consumers are present. So consumers are connected and acking or rejecting quickly, not stuck.
2. *Rates:* the deliver rate is huge, ack rate small, redelivery rate close to the deliver rate. So most deliveries are redeliveries.
3. *One message:* a peek or consumer log shows the same `message_id` redelivered with `x-delivery-count` 0. A delivery limit would have dead-lettered it once the count exceeded 20. A count stuck at 0 rules out `reject` and consumer crashes, which raise it, and is consistent with `nack`. A consumer timeout or a partition would also leave it unchanged, so step 4 checks the code.
4. *Code:* the handler catches every exception and nacks with requeue.
5. *Hypothesis tested:* one message with a malformed body fails every time and loops; good messages behind it are delayed.
6. *Fix and acceptance criteria:* use `reject` for retryable failures and `requeue=False` for non-retryable ones; after deploy, the redelivery rate falls, the bad message appears in the dead-letter queue with reason `delivery_limit` or `rejected`, and backlog drains.

Note that the recovery is a code change with a measurable acceptance criterion, and the poisoned message is preserved for inspection, not purged.

6.5 Recovery after partial failure

- **Dead letters** are data. Inspect a sample, fix the cause, then replay deliberately. The lab's step 7 peeks through the management API's "get messages" with requeue; observed on 4.4.0, repeated peeks did not change `x-delivery-count`. Inspecting with a consumer that disconnects without acking does increment it, and a dead-letter queue is itself a quorum queue with the default limit of 20 and usually no dead-letter exchange of its own: enough inspections would silently drop the evidence. The lab's `dead-letters` policy sets `delivery-limit` to -1 on `jobs.dead` for this reason.

- **Purging** a queue deletes its ready messages irreversibly. It is sometimes right (a backlog of obsolete cache invalidations); it needs a decision about what those messages were.
- **A quorum queue without a majority** recovers when enough members return. Forcing membership changes on a queue without quorum (for example deleting unreachable members) risks losing confirmed messages; prefer restoring the nodes. Treat any agent proposal to "reset" a node or delete its data directory as a data-loss decision.
- **An unknown publish outcome** is resolved by retrying with the same `message_id` and relying on consumer idempotency, not by guessing.

6.6 Exercise 7: diagnose from output

A worker service reports "stopped processing at 14:00 without errors". The process is alive. At 14:20 you capture:

```
name          messages_ready  messages_unacknowledged  consumers
renders       412             0                        0
```

Broker logs at 14:00 mention a consumer of `renders` being cancelled. The `renders` queue is a quorum queue; render jobs normally take 5–40 minutes. State your leading hypothesis, the evidence that supports it, one observation that would refute it, and two design options.

7 Keeping it safe and maintainable

7.1 Users, vhosts and permissions

RabbitMQ authorises each operation against three regular expressions per user per vhost: **configure** (create or delete exchanges and queues), **write** (publish to an exchange, bind to a queue) and **read** (consume from a queue, bind from an exchange). Access control. The lab's definitions grant:

```
"permissions": [
  {"user": "admin", "vhost": "jobs", "configure": ".*", "write":
   ↪  ".*", "read": ".*"},
  {"user": "app", "vhost": "jobs", "configure": "^$", "write":
   ↪  "^jobs$", "read": "^jobs\\.work$"}
```

```
]
```

`app` can publish to `jobs`, consume from `jobs.work`, and nothing else: it cannot declare queues (observed: 403), publish to `jobs.dlx` or read `jobs.dead`. In production, give each service its own user, scope permissions to the names it needs, and put unrelated applications in separate vhosts. One shared all-powerful user turns every service bug into a broker-wide incident.

The built-in `guest` user exists only on a node that boots without definitions, and by default it can connect only from loopback. Do not lift that restriction (`loopback_users = none`); create real users. The lab never creates `guest`, because it imports definitions at boot.

7.2 Secrets

The lab deliberately keeps lab-only passwords in `common.py` and password hashes in `definitions.json`. In a real deployment:

- Client credentials come from the secret store your deployment uses, not the repository.
- A definitions file contains password hashes. RabbitMQ's default hashing is a single salted SHA-256, cheap to attack offline (my assessment, not a RabbitMQ statement), so treat exported definitions as secret.
- The **Erlang cookie** gives full control of every node to anyone who can reach the inter-node and EPMD ports (25672 and 4369). It must be identical on all nodes, readable only by the RabbitMQ user (mode 600), and those ports must not be reachable from client networks. The lab cluster passes it through `RABBITMQ_ERLANG_COOKIE`; 4.4.0 honoured that and logged a warning that it was overriding the cookie from the environment. In production, put it in the cookie file through configuration management. Clustering, networking.

7.3 TLS and exposure

TLS is not enabled by default. The conventional ports are 5671 for AMQP over TLS and 15671 for the management UI over HTTPS. Anything crossing a network you do not fully control should use TLS

with certificate verification on the client. The management UI and HTTP API (15672) carry administrative power; expose them only on an operations network. The lab publishes every port on loopback only. TLS.

A container detail belongs here because it is a plausible generated mistake. The official image starts as root and drops to user `rabbitmq` (uid 999) to run the server, but `docker compose exec` and healthchecks run as root. Observed: when a root `rabbitmq-diagnostics` probe ran on a fresh volume before the server had created its Erlang cookie, the CLI created `/var/lib/rabbitmq/.erlang.cookie` as root with mode 0400, and the server then failed to start with `eacces` reading it. Whether this happens depends on which process wins the race, so it can pass once and fail on the next fresh start. The lab compose files run the container as `999:999`, so probes and the server share an identity. Watch for this pattern in agent-written healthchecks.

7.4 Reading an inherited broker

When you inherit a RabbitMQ deployment, establish what exists before changing it:

```
rabbitmqctl export_definitions /tmp/defs.json   # topology,
 ↪  users, policies
rabbitmqctl list_policies -p VHOST
rabbitmqctl list_operator_policies -p VHOST
rabbitmqctl list_queues -p VHOST name type messages_ready
 ↪  messages_unacknowledged consumers
rabbitmq-queues quorum_status QUEUE --vhost VHOST
rabbitmq-diagnostics status
```

Compare the export with whatever the repository says the topology is. Drift means someone, or some service, declares things at runtime. Note operator policies: they cap values set elsewhere and are easy to miss. Do not assume that re-importing the repository's definitions undoes drift.[1] Definitions.

[1]The definitions guide says a file imported at boot "will not overwrite anything already in the broker". It is not clear whether that holds for every object type, such as users and policies, or for a later `rabbitmqctl import_definitions`. Test on a lab broker before relying on either behaviour.

7.5 Exercise 8: review a definitions change

An agent proposes, for a new reporting service:

```
{"user": "reports", "vhost": "jobs", "configure": ".*", "write":
 ↪  ".*", "read": ".*"}
```

plus a policy `{"pattern": ".*", "definition": {"message-ttl": 3600000}}` "so old messages do not pile up". What does each change do to the existing pipeline, and what would you approve instead?

8 Integrating C++ clients (optional)

Read this if C++ services will publish or consume alongside Python ones. It needs chapters 2 and 4. None of the C or C++ clients is officially supported by the RabbitMQ team; they are community libraries listed on the client libraries page. Examples in this chapter are **untested excerpts** that show the shape of the code, not runnable programs.

8.1 The message contract comes first

RabbitMQ does not inspect bodies. Two languages can only interoperate through a contract you write down and both sides test:

- **Body encoding and schema**, with a version field or header.
- `content_type` (and `content_encoding` if compressed).
- `message_id` with defined semantics: the idempotency key, stable across retries.
- `delivery_mode`: persistent for anything that must survive a restart.
- **Headers** used for routing or tracing, with their AMQP types. Headers are typed tables; a C++ client that writes a header as a 64-bit integer and a Python consumer that expects a string will disagree silently.
- **Exchange, routing keys and queues** each side may use.
- **Failure semantics**: which errors a consumer rejects without requeue, and which it retries.

The broker semantics from chapters 2 and 4 are client-independent:

confirms, manual acks after the effect commits, prefetch, the reject-versus-nack delivery-count rule, and idempotent processing.

8.2 Library choices

- **rabbitmq-c** (C).[2] Its README says a socket, connection state or channel cannot be shared between threads: use one connection per thread or add your own locking. It is a synchronous, low-level API, and its README does not describe publisher confirms; expect to put the channel in confirm mode and read the acks from the connection yourself, and check the headers of the version you adopt. rabbitmq-c.
- **AMQP-CPP** (C++17). It is asynchronous: the library never blocks, and you integrate its socket with an event loop (libev, libuv and libevent handlers are provided). `AMQP::Reliable` wraps a channel to track confirms per publish with `onAck`/`onNack`/`onLost`; `AMQP::Throttle` caps outstanding publishes. Consumers call `channel.ack(deliveryTag)` and set prefetch with `channel.setQos(n)`. The README does not claim thread safety; keep each connection on one event-loop thread. AMQP-CPP.

AMQP 1.0 is an alternative protocol for new clients; the RabbitMQ team's supported AMQP 1.0 clients do not include C or C++. A generic AMQP 1.0 library such as Qpid Proton C++ would work in principle; that was not verified for this tutorial.

8.3 The shape of a correct C++ consumer

Untested AMQP-CPP excerpt, assuming an event loop and a connected `channel`:

```
channel.setQos(10);
channel.consume("jobs.work")
    .onReceived([&](const AMQP::Message &message, uint64_t tag,
     ↪  bool redelivered) {
```

[2] At the time of writing, the latest releases appeared to be rabbitmq-c v0.18.1 (October 2026) and AMQP-CPP v4.3.27 (October 2024), which would make AMQP-CPP the slower-moving of the two. These dates were not independently confirmed; check each project's releases before choosing.

```
        // Hand the work off; never block the event-loop thread.
        pool.submit([&, tag, id = message.messageID(),
                     body = std::string(message.body(),
↪   message.bodySize())] {
            bool ok = apply_with_idempotency_key(id, body);   //
             ↪  commits effect + key
            loop.post([&, tag, ok] {
↪   // back on the loop thread
                if (ok) channel.ack(tag);
                else    channel.reject(tag, AMQP::requeue);
                 ↪  // counts towards the limit
            });
        });
    });
```

The points to review are the same as in pika, made sharper by threads:

- The event-loop thread must not block, or heartbeats stop (the same failure as a blocking pika callback).
- The ack or reject happens on the channel's own thread, after the effect commits. Calling `channel.ack` directly from the pool thread is a data race in a library that does not promise thread safety.
- `reject` rather than a nack for retries, for the delivery-count reason in chapter 4.
- Lambdas that capture by reference must not outlive what they capture; shutdown must stop the pool before the channel is destroyed.

8.4 Exercise 9: review a C++ producer

A C++ service publishes with AMQP-CPP using a plain channel (not `AMQP::Reliable`), sets `messageID` to a fresh UUID on each send attempt, and writes the job number as a 64-bit integer header `job`. The Python worker reads `job` from the body. List the contract and reliability problems.

Worked answers

Exercise 1

`invoice.eu.created` goes to `audit` (`#`) and `eu` (`*.eu.#` with `#` matching one word); `billing`'s `invoice.*` needs exactly two words. `invoice.paid` goes to `audit` and `billing`. `refund.eu` goes to `audit` and `eu` (`#` matches zero words). `invoice` goes to `audit` only. `invoce .paid` matches `audit`'s `#`, so it *is* routed, to `audit` only; the producer sees a normal confirm. The typo is not detected because a catch-all binding exists: `mandatory` would not help either. Detecting it needs a contract test or monitoring of unexpected keys. The earlier trap (unroutable means confirmed and dropped) applies only when no binding matches at all.

Exercise 2

Jobs 2, 3, 4 and 5 return to the queue, all with `redelivered=True`; on a quorum queue their `x-delivery-count` is 1, because the connection was lost. Job 1 was acked and does not return. Jobs 3–5 had never been processed, but still carry the flag. Job 2's effect committed, so the second worker must recognise its `message_id` in the same store and ack without repeating the effect. It must not use `redelivered` to make that decision.

Exercise 3

Right: confirm mode, and retrying after connection failure. Problems:

- No `mandatory`, so a wrong routing key or missing binding is confirmed and lost.
- No `delivery_mode=2`; on a classic queue the message is not persistent.
- No `message_id`; a retry after an unknown confirm creates a duplicate that consumers cannot recognise.
- The retry is unbounded and immediate: a broker without quorum or under an alarm yields a hot loop or an indefinite wait. It also needs a bounded confirm wait.
- `AMQPError` also covers channel errors such as 403 or 406, which no retry fixes.

Ask for: a stable `message_id`, `mandatory` with handling, persistence, backoff with a limit, and a test that kills the broker connection mid-batch and shows every job present once after consumer deduplication.

Exercise 4

With prefetch 1, the first worker recorded jobs 1–4 and acked 1–3; only job 4 was in flight. The second worker prints `job-4 redelivered=True count=1 duplicate ignored`, then jobs 5–10 with `redelivered=False count=0 recorded`. With prefetch 10, jobs 4–10 had all been delivered to the first worker, so the second worker sees job 4 as a duplicate and jobs 5–10 as `redelivered=True count=1 recorded`. The output differs, the outcome does not: this is why `redelivered` cannot be the deduplication signal.

Exercise 5

- `queue_declare("thumbnails", arguments=...)` uses pika's defaults (`durable=False`, `exclusive=False`). On 4.3+ this transient non-exclusive queue is denied, and the connection closes with 541. Even on an old broker it would be a non-replicated classic queue lost on restart. Require: topology owned by definitions, or an explicit durable quorum queue.
- The dead-letter exchange is a hard-coded argument; move it to a policy.
- `prefetch_count=0` means unlimited; one worker hoards the backlog. Choose a measured value.
- As declared, `thumbnails` is a classic queue, which has no delivery limit at all, so any requeue-on-failure loops forever. Once it is a quorum queue, `nack(requeue=True)` still never increments the delivery count, so the limit never fires and nothing reaches the dead-letter exchange: a hot loop. Require `reject` for retryable errors, `requeue=False` for permanent ones, and backoff if failures are transient downstream outages.
- `make_thumbnail` runs on the connection thread; a long thumbnail blocks heartbeats. Require the worker-thread pattern or a bound on job time.
- The handler has no idempotency key, so redeliveries repeat the work.

The agent's claim fails twice: the declared queue type has no delivery limit, and on a quorum queue the nack rule would bypass it.

Exercise 6

`check_if_node_is_quorum_critical` on `n2` fails (exit 69) and names `orders`: with members on `n1` and `n2`, stopping `n2` leaves one of two, which is not a majority. During the restart, publishes to `orders` connect but are not confirmed until `n2` returns; consumers receive nothing from it. First run `rabbitmq-queues grow rabbit@n3 all --queue-pattern '^orders$'` (with the right vhost pattern), confirm three members with `quorum_status`, then rerun the check. Also ask why `orders` had two members: it was probably declared while a node was down or before the cluster formed.

Exercise 7

Leading hypothesis: the quorum queue's consumer timeout (30 minutes by default) cancelled the worker's consumer while a long render was unacknowledged. The worker did not handle the cancellation, so the process lives but consumes nothing. Support: zero consumers, zero unacked, a cancel in the broker log, and job times that exceed 30 minutes. Refutation: if `consumers` were non-zero, or the cancel predated the first 40-minute job, or the worker logged a connection error instead, look elsewhere. The render that timed out was returned to the queue and may have been redelivered elsewhere, so check for duplicates. Options: raise the queue's `consumer-timeout` by policy above the longest legitimate job (with an alarm on job duration); handle consumer cancellation by re-subscribing or exiting so the supervisor restarts the worker; or restructure long jobs so the message is acked when the job is durably accepted by another system that tracks progress.

Exercise 8

The `reports` user can delete or redeclare every queue and read every message in the vhost, including `jobs.dead`. Approve a user with `configure: "^$"` and read and write limited to its own queue and exchange, or a separate vhost if it does not need `jobs` at all. The `.*`

policy matches every queue in the vhost, including `jobs.work` and `jobs.dead`. Only one policy applies to a queue, and at the default priority 0 it ties with `work-failures` on `jobs.work` and with `dead -letters` on `jobs.dead`, so the broker chooses nondeterministically on each. Where it won, `jobs.work` would lose its dead-letter and delivery-limit settings and drop pending jobs older than an hour silently; `jobs .dead` would regain the default delivery limit and discard dead letters, the evidence of past failures, after an hour. Approve a TTL policy only with a narrow pattern, a stated reason, and an explicit priority.

Exercise 9

No confirms (a plain channel), so lost publishes are invisible; use `AMQP::Reliable` and handle `onNack`/`onLost`. A fresh UUID per attempt defeats idempotency; the ID must be stable per job. The `job` header is not part of the contract the Python worker reads, and its type would have to be specified if it were. Also check `content_type`, persistence, `mandatory` or an alternate exchange, and a cross-language contract test that runs one producer against the other consumer.

Field reference

Need	Command or setting (4.4)
Queue state	`rabbitmqctl list_queues -p V name type messages_ready messages_unacknowledged consumers`
Quorum membership	`rabbitmq-queues quorum _status Q --vhost V`
Safe to stop a node?	`rabbitmq-queues check_if _node_is_quorum_critical` (exit 69 = no)
Add members	`rabbitmq-queues grow rabbit@NODE all --vhost -pattern P --queue-pattern P`

Need	Command or setting (4.4)
Alarms	`rabbitmq-diagnostics check_alarms`, `rabbitmq-diagnostics alarms` (not `check_local_alarms`)
Connection states	`rabbitmqctl list_connections user state`
Topology export	`rabbitmqctl export_definitions FILE`
Policies	`rabbitmqctl list_policies -p V`, `list_operator_policies -p V`
Drain for upgrade	`rabbitmq-upgrade drain`, `rabbitmq-upgrade await_online_quorum_plus_one`
Message tracing	`rabbitmqctl trace_on -p V` (firehose; costs performance; off at boot)
Effective setting	`rabbitmqctl eval 'application:get_env(rabbit, KEY).'`
Delivery limit	`delivery-limit` policy key; default 20; counts `reject` and crashes, not `nack`
At-least-once DLX	`dead-letter-strategy: at-least-once` + `overflow: reject-publish` + DLX
Consumer timeout	`consumer_timeout` (ms; 1,800,000 observed), `consumer-timeout` policy; quorum queues only
Ports	5672 AMQP, 5671 AMQPS, 15672 management, 15692 Prometheus, 25672 inter-node, 4369 EPMD, 5552 streams

Firehose, rabbitmq-diagnostics(8), rabbitmqctl(8).

Example files, sources and verification

Files

All under `rabbitmq/examples/`:

- `compose.yaml`, `broker/rabbitmq.conf`, `broker/definitions.json`: the single-node lab broker, loopback ports only, lab-only credentials.
- `requirements.txt` (pika 1.4.4), `common.py`, `producer.py`, `worker.py`: the pipeline.
- `lab.sh`: chapters 2–4 experiments; destroys and recreates the lab broker.
- `cluster/compose.yaml`, `cluster/rabbitmq.conf`, `cluster/lab.sh`: the three-node cluster and chapter 5 experiments; destroys and recreates the cluster lab.

The scripts change only the two Compose projects `rabbitmq-lab` and `rabbitmq-cluster-lab` and their volumes, plus `ledger.sqlite` and `nack.log` in `examples/`. Never point them at a real broker.

Versions and sources

Target and observed: RabbitMQ 4.4.0 (image `rabbitmq:4.4.0-management`, Erlang 29.1.2), pika 1.4.4, Python 3.12.3, Docker 27.2.0. Research date: 21 October 2026. Primary sources are linked beside each claim: the RabbitMQ documentation (mutable "docs" URLs, read on that date), release notes for 4.0.1, 4.3.0 and 4.4.0, pika's documentation and source, and the C and C++ client repositories.

What was and was not verified

`verification.md` records the exact runs. Every lab result quoted here was observed on 4.4.0 containers; `check_local_alarms` was also compared on 4.3.6 and 4.2.9. Not verified: the delayed-retry sketch, throughput and prefetch tuning, TLS, upgrades, true network partitions, hosts other than Docker on Linux, and all C++ code. C++ library claims come from their READMEs and release metadata.